

Laboratorium
Robotika dan Sistem Cerdas
Departemen Teknik Komputer
Institut Teknologi Sepuluh Nopember

Laporan Sementara
Praktikum Sistem Mikroprosesor dan Sistem Tertanam

Pengenalan Mikrokontroller

Athaya Khairani Adi - 5024241007

6 November 2026

1 Pendahuluan

1.1 Latar Belakang

Perkembangan teknologi sistem tertanam (embedded system) dan Internet of Things (IoT) mendorong penggunaan mikrokontroler sebagai komponen utama dalam berbagai perangkat elektronik cerdas. Mikrokontroler memungkinkan perangkat menjalankan instruksi, membaca data dari sensor, serta mengendalikan komponen elektronik secara otomatis. Penerapannya dapat ditemukan pada sistem otomasi industri, perangkat rumah pintar, sistem pemantauan lingkungan, hingga teknologi kendaraan. Oleh karena itu, pemahaman mengenai mikrokontroler menjadi salah satu kompetensi dasar dalam bidang teknik komputer.

Dalam implementasinya, pengembangan sistem berbasis mikrokontroler memerlukan pemahaman mengenai karakteristik perangkat keras, pemrograman, serta komunikasi antara mikrokontroler dan komponen eksternal. Beberapa permasalahan yang sering dijumpai meliputi pengaturan pin masukan dan keluaran (GPIO), ketidakstabilan pembacaan tombol akibat kondisi floating dan bouncing, serta pengolahan sinyal analog dari sensor menggunakan Analog-to-Digital Converter (ADC). Apabila konsep-konsep tersebut tidak dipahami dengan baik, sistem dapat menghasilkan pembacaan yang tidak akurat atau respons yang tidak sesuai dengan kondisi sebenarnya.

Praktikum ini membahas pengenalan dan pemrograman dua platform mikrokontroler, yaitu STM32 Blackpill dan ESP32, menggunakan framework Arduino dan ESP-IDF melalui Visual Studio Code dengan ekstensi PlatformIO. Selain itu, praktikum mencakup konfigurasi GPIO, penggunaan resistor pull-up dan pull-down, teknik debouncing, serta pembacaan sinyal analog menggunakan ADC. Materi tersebut memberikan pemahaman mengenai proses pengembangan sistem tertanam, mulai dari konfigurasi perangkat hingga pengolahan masukan.

Melalui praktikum ini, mahasiswa diharapkan mampu memahami prinsip kerja mikrokontroler, melakukan pemrograman pada platform yang berbeda, serta mengintegrasikan perangkat keras dengan perangkat lunak. Kompetensi tersebut menjadi landasan dalam pengembangan sistem otomatisasi dan aplikasi IoT yang membutuhkan akuisisi data sensor, pengendalian perangkat, dan pemrosesan informasi secara real-time.

1.2 Dasar Teori

1. **Mikrokontroler**
   Mikrokontroler adalah sebuah sistem komputer mini yang diintegrasikan dalam sebuah keping sirkuit terpadu (*integrated circuit* atau IC). Komponen utama yang membentuk mikrokontroler meliputi *Central Processing Unit* (CPU) sebagai otak pemrosesan instruksi, memori Flash sebagai media penyimpanan program secara non-volatil, serta *Random Access Memory* (RAM) untuk penyimpanan data sementara yang bersifat volatil. Selain itu, mikrokontroler dilengkapi dengan berbagai periferal pendukung seperti *timer*/pencacah, antarmuka komunikasi serial (UART, I2C, SPI), serta *Analog-to-Digital Converter* (ADC). Penggunaan mikrokontroler sangat dominan dalam sistem tertanam (*embedded system*) karena efisiensi daya yang tinggi, ukuran yang kompak, serta kemampuan kontrol *real-time* yang andal dengan kebutuhan perangkat keras eksternal yang minimal.

2. **Pin *General Purpose Input/Output* (GPIO)**

GPIO merupakan pin serbaguna pada mikrokontroler yang dapat dikonfigurasi melalui perangkat lunak untuk berinteraksi dengan dunia luar. GPIO memiliki tiga mode operasi utama:

- *Digital Output*: Berfungsi untuk menghasilkan level tegangan diskrit, yaitu logika *HIGH* (biasanya $3{,}3$ V atau 5 V) atau logika *LOW* (0 V), yang umumnya digunakan untuk mengendalikan aktuator seperti LED, relay, atau buzzer.
- *Digital Input*: Berfungsi untuk membaca kondisi logika diskrit dari perangkat eksternal seperti tombol (*push button*) atau sensor digital.
- *Analog Input*: Berfungsi untuk membaca sinyal tegangan kontinu yang kemudian dikonversi menjadi nilai digital melalui modul ADC.

3. **STM32 Blackpill dan *Framework* Arduino**
   STM32 Blackpill adalah papan pengembangan (*development board*) berkinerja tinggi yang berbasis pada mikrokontroler ARM Cortex-M4 32-bit STM32F401CC (atau varian F411CE), dengan frekuensi *clock* hingga 84 MHz, memori Flash sebesar 256 KB, dan RAM 64 KB. Papan ini menggunakan pemrogram eksternal seperti ST-Link melalui protokol *Serial Wire Debug* (SWD) yang menggunakan jalur komunikasi `SWDIO` dan `SWCLK`. Dalam praktikum ini, STM32 diprogram menggunakan *framework* Arduino melalui port STM32duino, yang menyediakan fungsi-fungsi standar seperti `pinMode()`, `digitalWrite()`, dan `delay()`. STM32duino memanfaatkan lapisan *Hardware Abstraction Layer* (HAL) dari STMicroelectronics untuk memberikan abstraksi perangkat keras yang andal dan efisien.

4. **ESP32 dan *Framework* ESP-IDF**
   ESP32 adalah mikrokontroler sistem-pada-cip (*System on Chip* / SoC) 32-bit buatan Espressif Systems yang terintegrasi penuh dengan modul Wi-Fi 802.11 b/g/n dan Bluetooth (Classic serta BLE). Papan ESP32 DevKit umumnya dilengkapi dengan cip konverter USB-ke-Serial seperti CP2102 atau CH340 yang memungkinkan pemrograman langsung melalui kabel USB. ESP32 mendukung dua pendekatan pengembangan utama:

   - *Framework* Arduino: Menggunakan fungsi `setup()` untuk inisialisasi sistem dan `loop()` untuk eksekusi berulang, menawarkan kemudahan dan kecepatan pengembangan.
   - *Framework* ESP-IDF (*Espressif IoT Development Framework*): Merupakan kerangka kerja resmi yang dibangun di atas sistem operasi *real-time* FreeRTOS, memberikan kontrol tingkat rendah yang sangat terperinci terhadap manajemen tugas (*task*), sinkronisasi, dan periferal perangkat keras.

5. **Kondisi *Floating* dan Resistor *Pull-up*/*Pull-down***
   Kondisi *floating* adalah keadaan di mana pin input digital tidak terhubung ke referensi tegangan yang pasti (baik ke sumber tegangan maupun ke tanah/GND), sehingga nilai logika yang dibaca menjadi tidak menentu dan rentan terhadap induksi derau elektromagnetik (*noise*). Untuk mengatasi hal ini, digunakan resistor *pull-up* atau *pull-down*:

   - Resistor *Pull-up*: Menghubungkan pin input ke jalur tegangan positif ($3{,}3$ V) melalui sebuah resistor. Saat tombol tidak ditekan, pin terbaca *HIGH*; saat ditekan, pin terhubung ke GND dan terbaca *LOW*.

- Resistor *Pull-down*: Menghubungkan pin input ke GND melalui sebuah resistor. Saat tombol tidak ditekan, pin terbaca *LOW*; saat ditekan, pin terhubung ke sumber tegangan dan terbaca *HIGH*.

Resistor ini dapat diaktifkan secara internal melalui konfigurasi perangkat lunak pada sebagian besar pin mikrokontroler. Namun, khusus pada ESP32, pin GPIO 34 hingga 39 tidak dilengkapi dengan resistor *pull-up* atau *pull-down* internal, sehingga wajib menggunakan resistor eksternal secara fisik pada rangkaian.

6. ***Bouncing* dan *Debouncing***
   Bouncing adalah fenomena fisik transien yang terjadi pada sakelar mekanis atau tombol, di mana kontak logam saling memantul dalam skala milidetik saat ditekan atau dilepas. Hal ini menyebabkan fluktuasi sinyal tegangan yang beralih antara *HIGH* dan *LOW* secara cepat sebelum akhirnya stabil. Mikrokontroler yang sangat cepat dapat keliru mengartikan fluktuasi ini sebagai beberapa kali penekanan tombol. Teknik *debouncing* diperlukan untuk menyaring sinyal tersebut, yang dapat dilakukan melalui dua cara:

   - *Software Debouncing*: Menggunakan fungsi pewaktuan seperti `millis()` untuk memeriksa kestabilan status sinyal setelah jeda waktu tertentu (misalnya 50 ms).
   - *Hardware Debouncing*: Menggunakan rangkaian tapis RC (*resistor-capacitor*) atau IC *Schmitt trigger* untuk meredam lonjakan tegangan transien secara fisik.

7. ***Analog-to-Digital Converter* (ADC) dan Sensor LDR**
   Analog-to-Digital Converter (ADC) adalah komponen penting yang mengubah sinyal tegangan analog kontinu menjadi representasi nilai digital diskrit agar dapat diproses oleh CPU mikrokontroler. Proses konversi ADC melibatkan tiga tahap utama: pencuplikan (*sampling*), kuantisasi (*quantization*), dan pengodean (*encoding*). Resolusi ADC ditentukan oleh jumlah bit (n), di mana total tingkat kuantisasi dinyatakan dengan persamaan:

$$\text{Tingkat Kuantisasi} = 2^n$$

   Semakin tinggi nilai bit resolusi n, semakin teliti ADC dalam membedakan variasi tegangan masukan. Sebagai contoh, ESP32 dilengkapi dengan ADC beresolusi 12-bit, yang menghasilkan rentang nilai digital diskrit dari 0 hingga $2^{12} - 1 = 4095$ melalui fungsi `analogRead(pin)`. Pembacaan sensor cahaya seperti *Light Dependent Resistor* (LDR) memanfaatkan prinsip pembagi tegangan, di mana resistansi LDR berbanding terbalik dengan intensitas cahaya yang mengenainya:

$$V_{\text{out}} = V_{\text{in}} \times \frac{R_{\text{fixed}}}{R_{\text{fixed}} + R_{\text{LDR}}}$$

   Perubahan tegangan keluaran V_{out} ini kemudian diakuisisi oleh pin ADC mikrokontroler untuk mengukur tingkat intensitas cahaya secara kuantitatif.

2 Tugas Pendahuluan

1. Jelaskan perbedaan mendasar antara framework Arduino dan ESP-IDF dalam memprogram mikrokontroler, termasuk kelebihan dan kekurangan masing-masing.

Jawaban = Framework Arduino dan ESP-IDF memiliki cara kerja yang berbeda dalam pengembangan program pada ESP32. Arduino menyediakan fungsi-fungsi sederhana, seperti `setup()`, `loop()`, dan `digitalWrite()`, sehingga pengguna tidak perlu mengatur setiap detail perangkat keras secara langsung. Hal ini membuat Arduino lebih mudah dipahami dan digunakan, terutama bagi pemula.

Sementara itu, ESP-IDF memberikan akses yang lebih luas terhadap fitur perangkat keras ESP32 dan pengelolaan tugas menggunakan sistem operasi *real-time* FreeRTOS. Program pada ESP-IDF dijalankan melalui fungsi utama `app_main()`. Dengan pendekatan ini, pengguna memiliki kendali yang lebih besar dalam mengatur kinerja dan penggunaan sumber daya perangkat.

Berikut perbandingan kelebihan dan kekurangan kedua framework tersebut:

Tabel 1: Perbandingan kelebihan dan kekurangan framework Arduino dan ESP-IDF

Aspek	**Arduino**	**ESP-IDF**
Kelebihan	Lebih mudah dipelajari, struktur kode sederhana, memiliki banyak pustaka dan dukungan komunitas, serta dapat digunakan pada berbagai jenis papan mikrokontroler.	Memberikan akses lebih luas terhadap fitur perangkat keras ESP32 dan FreeRTOS, memungkinkan pengaturan kinerja dan penggunaan daya secara lebih terperinci, serta cocok untuk pengembangan aplikasi IoT yang kompleks.
Kekurangan	Beberapa detail perangkat keras tidak ditampilkan secara langsung sehingga pengguna memiliki keterbatasan dalam melakukan pengaturan tingkat lanjut.	Membutuhkan pemahaman yang lebih mendalam, memiliki struktur program yang relatif kompleks, serta penggunaannya lebih berfokus pada perangkat mikrokontroler keluaran Espressif.

2. Mengapa board STM32 Blackpill memerlukan programmer eksternal (ST-Link) untuk di-flash, sedangkan ESP32 dapat diprogram langsung melalui kabel USB? Kaitkan jawaban Anda dengan keberadaan chip USB-to-Serial.

   Jawaban = ESP32 DevKit memiliki cip *USB-to-Serial* (CP2102/CH340) yang menerjemahkan data USB dari komputer menjadi sinyal UART. *Bootloader* serial pada ROM ESP32 menerima program lewat UART tersebut, sehingga satu kabel USB sudah cukup. Sebaliknya, Blackpill tidak memiliki cip *USB-to-Serial* maupun programmer bawaan, sehingga tidak ada perangkat di papan yang menerjemahkan data dari komputer. ST-Link berperan sebagai jembatan eksternal yang menerjemahkan perintah USB menjadi sinyal SWD (SWDIO dan SWCLK) untuk menulis memori *flash* sekaligus mendukung *debugging*. Secara teknis, STM32F401 juga dapat diprogram lewat *bootloader* bawaan (UART atau USB DFU), tetapi memerlukan pengaturan pin BOOT0 dan tidak mendukung *debugging*.

3. Jelaskan perbedaan kondisi logika (HIGH/LOW) pada konfigurasi pull-up dan pull-down resistor, baik saat tombol tidak ditekan maupun saat ditekan.

   Jawaban = Pada *pull-up*, pin dihubungkan ke 3,3 V melalui resistor dan tombol dipasang ke GND. Pada *pull-down*, pin dihubungkan ke GND melalui resistor dan tombol dipasang ke 3,3 V. Keduanya mencegah kondisi *floating*.

Tabel 2: Kondisi logika pin pada konfigurasi *pull-up* dan *pull-down*

Konfigurasi	Tidak ditekan	Ditekan	Logika aktif
Pull-up	HIGH	LOW	Aktif LOW
Pull-down	LOW	HIGH	Aktif HIGH

Saat tombol ditekan, resistor membatasi arus agar tidak terjadi hubungan singkat, yaitu $3{,}3\,\mathrm{V}/10\,\mathrm{k\Omega} = 0{,}33\,\mathrm{mA}$.

4. Jelaskan fenomena bouncing pada tombol mekanik, mengapa hal ini bisa terjadi, dan sebutkan minimal dua pendekatan untuk mengatasinya.

   Jawaban = *Bouncing* adalah munculnya deretan transisi HIGH-LOW yang tidak diinginkan selama beberapa milidetik saat kontak tombol tertutup atau terbuka. Hal ini terjadi karena kontak logam bersifat lentur dan bergetar (memantul) sebelum menempel diam. Mikrokontroler membaca pin jauh lebih cepat daripada durasi pantulan, sehingga satu penekanan dapat terbaca sebagai beberapa kali penekanan. Pendekatan penanganannya:

   - **Perangkat lunak:** status baru dianggap sah setelah stabil melebihi selang waktu tertentu (misalnya 50 ms) menggunakan `millis()` yang tidak memblokir program, atau dengan `delay()` yang lebih sederhana tetapi memblokir program.
   - **Perangkat keras:** tapis lolos rendah RC ($\tau = RC$) yang dipadukan dengan *Schmitt trigger*, atau rangkaian *latch* SR.

5. Bagaimana ADC (Analog-to-Digital Converter) mengubah sinyal analog menjadi nilai digital? Jelaskan pula pengaruh resolusi bit ADC terhadap presisi hasil pembacaan.

   Jawaban = ADC bekerja dalam tiga tahap: (1) pencuplikan dan penahanan tegangan masukan, (2) kuantisasi, yaitu membagi rentang V_{ref} menjadi 2^n tingkat dan memetakan cuplikan ke tingkat terdekat, dan (3) pengodean tingkat tersebut menjadi bilangan biner. Hubungannya:

$$D = \left\lfloor \frac{V_{\text{in}}}{V_{\text{ref}}} \times 2^n \right\rfloor, \qquad \text{LSB} = \frac{V_{\text{ref}}}{2^n} \tag{1}$$

   Makin besar jumlah bit n, makin kecil LSB sehingga perubahan tegangan yang lebih halus dapat dibedakan. Pada $V_{\text{ref}} = 3{,}3\ \mathrm{V}$, ADC 10 bit memiliki LSB sekitar 3,22 mV, ADC 12 bit (ESP32) sekitar 0,81 mV, dan ADC 16 bit sekitar 50,35 μV. Resolusi tinggi meningkatkan presisi, tetapi belum tentu meningkatkan akurasi karena akurasi juga dipengaruhi derau dan ketidaklinearan ADC.

6. Lakukan set-up untuk tools: vscode, ekstensi platform-io (dengan framework esp-idf dan stm32), dan ekstensi serial-plotter pada laptop kalian, berikan screenshot bahwa laptop kalian sudah terset-up dengan baik.

   Jawaban = Penyiapan dan pengujian lingkungan pengembangan dilakukan melalui langkah-langkah berikut:

   (a) **Instalasi Awal dan Pemeriksaan Platform**

   i. Memasang *Visual Studio Code* dan ekstensi *PlatformIO IDE* melalui panel *Extensions*, lalu memulai ulang VSCode hingga panel dan ikon PlatformIO muncul (Gambar 1).

ii. Memeriksa menu *PIO Home* → *Platforms* untuk memastikan platform pengembangan *Espressif 32* dan *ST STM32* sudah terpasang dengan baik (Gambar 2).

(b) **Pemasangan Ekstensi Serial Plotter**

i. Membuka panel *Extensions* (*Ctrl*+*Shift*+*X*) dan mencari *Serial Plotter* buatan Mario Zechner.

ii. Mengklik tombol *Install* pada ekstensi tersebut agar siap digunakan untuk visualisasi data serial (Gambar 3).

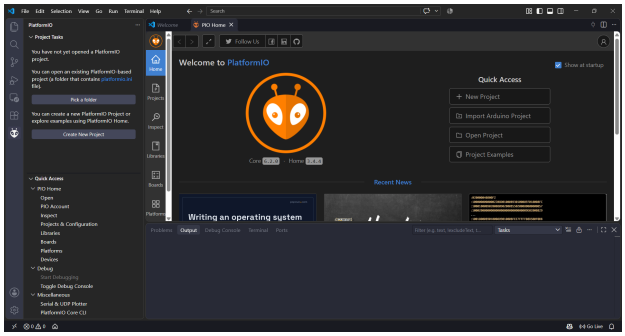
(c) **Konfigurasi Proyek dan Instalasi Paket (Package) ESP32**

i. Menjalankan perintah terminal `pio pkg install -e esp32dev` untuk mengunduh dan memasang dependensi serta pustaka ESP-IDF secara otomatis (Gambar 4).

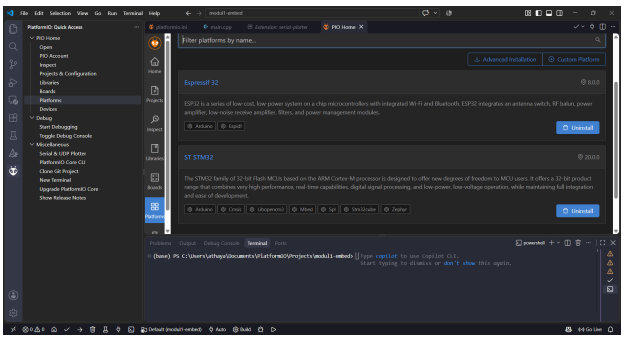
(d) **Uji Build ESP32 dan STM32**

i. Menyiapkan kode sumber *Blink* berbasis ESP-IDF (`main.c`) pada folder *src*, lalu melakukan *Build* hingga terminal menunjukkan status `SUCCESS` (Gambar 5).

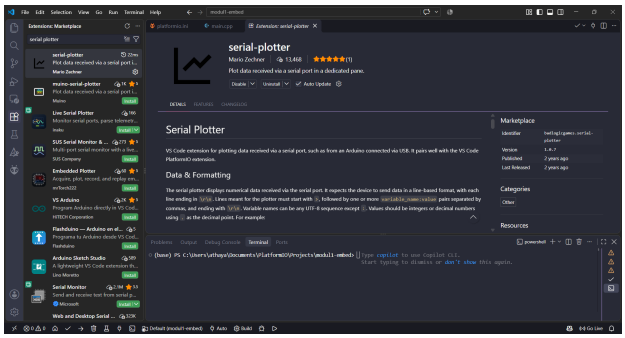
ii. Mengganti konfigurasi lingkungan ke STM32 Blackpill pada `platformio.ini`, membuat kode *Blink* Arduino (`main.cpp`), lalu menjalankan *Build* hingga sukses (Gambar 6).



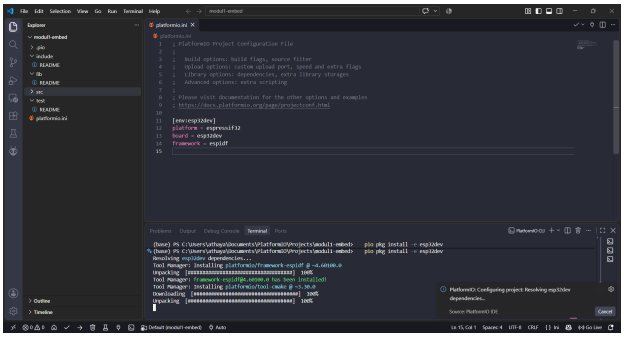
Gambar 1: Tampilan awal *PIO Home* di VSCode



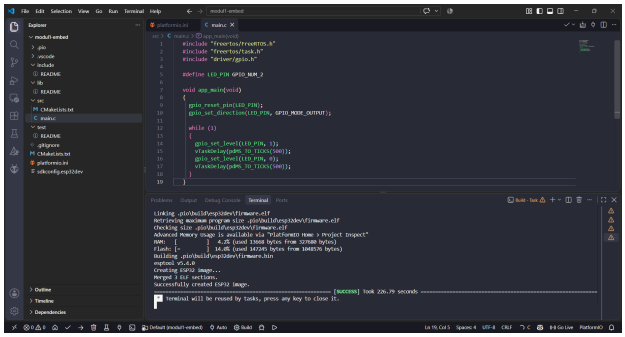
Gambar 2: Platform *Espressif 32* dan *ST STM32* terpasang



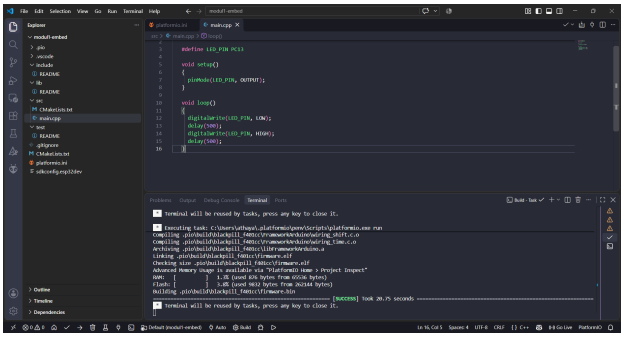
Gambar 3: Pemasangan ekstensi *Serial Plotter*



Gambar 4: Eksekusi perintah `pio pkg install -e esp32dev`



Gambar 5: Build sukses ESP32 dengan ESP-IDF



Gambar 6: Build sukses STM32 Blackpill